

GateKeeper

SimPy Tabanlı Ağ Geçidi Trafik ve
Güvenlik Simülasyonu

Benzetim Programları Dersi

Mersin Üniversitesi
Bilişim Sistemleri ve Teknolojileri (CTIS) Bölümü

Hazırlayan:

Ahmet Feyzi Gülmez — 21430070039

Emircan Altuntaş — 22430070020

Ahmet Duran Gökçe — 21430070043

GitHub: <https://github.com/emircanaltuntas/GateKeeper>

2025 - 2026 Akademik Yılı

1. Giriş

Bu projede bir ağ geçidi (gateway) üzerinden geçen trafiğin simülasyonunu yapmayı amaçladık. Konuyu seçme nedenimiz aslında basit: ağ derslerinde hep teorik olarak gördüğümüz kuyruk yapıları, paket önceliklendirme ve drop mekanizmaları gibi kavramları somut bir şekilde görmek istedik. Yani "kuyruk dolunca ne olur?" sorusuna sadece formülle değil, çalışan bir simülasyonla cevap vermek istedik.

Proje üç kişilik ekibimiz tarafından geliştirildi. Ahmet Feyzi daha çok simülasyon motoruyla, Emircan arayüz tarafıyla, Ahmet Duran da raporlama ve test kısmıyla ilgilendi. Tabi herkes her şeye az çok el attı, net bir ayırım yok.

2. Problem Tanımı

Gerçek hayatta bir ağ geçidi sınırlı kapasiteye sahiptir. Aynı anda işleyebileceği paket sayısı bellidir ve kuyrukta bekleyebilecek paket sayısının da bir üst limiti vardır. Trafik yoğunlaştığında bazı paketlerin düşürülmesi (drop edilmesi) kaçınılmaz hale gelir. Buradaki asıl soru şu: hangi paketler düşürülecek?

Eğer rastgele veya sırayla düşürürseniz, kritik bir yönetim paketi de gidebilir. Bu yüzden öncelik tabanlı bir mekanizma şart. Projemizde tam olarak bunu modelliyoruz: farklı öncelik seviyelerine sahip paketlerin (admin, normal, şüpheli) sınırlı kapasiteli bir geçitten geçerken nasıl davrandığını simüle ediyoruz. Katmanlı bir drop politikası uygulayarak kritik trafiğin korunup korunmadığını test ediyoruz.

3. Kullanılan Veri Seti

Projede hazır bir veri seti kullanmadık. Bunun yerine paket trafiğini stokastik olarak kendimiz üretiyoruz. Her paket türünün geliş aralığı üstel dağılıma (exponential distribution) göre belirleniyor. Bunu tercih etmemizin sebebi, gerçek ağ trafiğinin de büyük ölçüde bu dağılıma uyması. Deterministik (sabit aralıklı) üretim yapsaydık sonuçlar yapay olurdu.

Üretilen paket türleri ve parametreleri şu şekilde:

Paket Türü	Öncelik	Ort. Geliş Aralığı	Açıklama
Admin	0 (en yüksek)	8.0 birim	Yönetici komutları, asla drop edilmez
Normal	1	2.0 birim	Standart kullanıcı trafiği
Şüpheli	2 (en düşük)	1.2 birim	Potansiyel tehdit, ilk drop edilen

Bunların dışında sunucu kapasitesi 3, kuyruk sınırı 10, simülasyon süresi 200 birim zaman olarak belirlendi. Şüpheli paketler için drop eşiği kuyruk > 10, normal paketler için kuyruk > 13 şeklinde ayarlandı.

4. Kod Tarafı ve Kullanılan Kütüphaneler

Projenin simülasyon motoru tamamen Python ile yazıldı. Ana kütüphanemiz SimPy. SimPy, ayrık olay simülasyonu (Discrete Event Simulation) için geliştirilmiş bir Python kütüphanesi. Biz bunu seçtik çünkü kuyruk ve kaynak paylaşımı problemleri için biçilmiş kaftan.

Kodda kullandığımız temel kütüphaneler:

- SimPy: Simülasyon motoru. PriorityResource yapısını kullanarak öncelikli kuyruk yönetimi sağlıyoruz. Düşük öncelik değeri = yüksek öncelik anlamına geliyor, yani admin paketleri (öncelik=0) her zaman öne geçiyor.
- random: Paket geliş aralıklarını üstel dağılımla üretmek için random.expovariate fonksiyonunu kullanıyoruz.
- statistics: Ortalama bekleme süresi, drop oranı gibi metrikleri hesaplamak için.
- Streamlit: Arayüz tarafı için (bir sonraki bölümde detaylı anlatılıyor).
- Plotly: Grafik ve görselleştirmeler için.

Kodda iki ana sınıf var: AgGecidi sınıfı sunucu kaynağını ve istatistik sayaçlarını tutuyor, PaketUretici sınıfı ise her paket türü için bağımsız üretim süreçlerini yönetiyor. Kuyruk doluluk takibi de ayrı bir SimPy process olarak arka planda çalışıyor. Drop kararı şu mantıkla veriliyor: kuyruk 10'u geçtiyse şüpheli paketi düşür, 13'ü geçtiyse normal paketi de düşür, admin paketine dokunma.

5. Arayüz Teknolojisi

Arayüz için Streamlit tercih ettik. Bunda birkaç sebep var: birincisi Python ekosistemiyle doğrudan entegre, yani SimPy kodumuzu ayrı bir API'ye sarmamıza gerek kalmıyor. İkincisi, Streamlit'in slider, selectbox gibi widget'ları sayesinde kullanıcı simülasyon parametrelerini kolayca değiştirebiliyor. Üçüncüsü de deploy etmesi kolay, Streamlit Cloud üzerinden paylaşabiliyoruz.

Arayüzde kullanıcıya sunduğumuz kontroller:

- Sunucu kapasitesini ayarlama (1-10 arası)
- Kuyruk sınırını değiştirme
- Paket geliş oranlarını slider ile ayarlama
- Simülasyon süresini belirleme
- Seed değerini girme (tekrarlanabilirlik için)

Kullanıcı parametreleri ayarlayıp "Simülasyonu Başlat" butonuna bastığında, arka planda SimPy çalışıyor ve sonuçlar anlık olarak ekrana yansıyor.

6. Görselleştirme ve Canlı İzleme

Projenin görselleştirme kısmına özellikle önem verdik çünkü simülasyonun çıktısı sadece rakamlardan ibaret olursa anlaması zor oluyor. Bu yüzden birkaç farklı grafik ve görsel yapı entegre ettik:

6.1. Kuyruk Doluluk Grafiği

Zamana bağlı kuyruk doluluk grafiği, simülasyon boyunca kuyruğun nasıl dalgalandığını gösteriyor. Plotly ile çizdiğimiz bu grafik sayesinde hangi zaman dilimlerinde drop mekanizmasının devreye girdiği net bir şekilde görülebiliyor. Eşik çizgileri de grafiğe eklendi.

6.2. Isı Haritası (Heatmap)

Paket türlerine göre zaman dilimlerindeki yoğunluğu gösteren bir ısı haritası oluşturduk. Bu harita, hangi zaman aralığında hangi tür paketin sistemi ne kadar meşgul ettiğini renk skalasıyla gösteriyor. Plotly'nin imshow fonksiyonuyla oluşturduğumuz bu harita, özellikle burst dönemlerini tespit etmekte çok faydalı oldu.

6.3. Drop Oranı Pasta Grafiği

Her paket türünün toplam drop içindeki payını gösteren basit ama etkili bir pasta grafiği. Admin paketlerinin sıfır dropla işlendiğini, şüphelilerin ise en büyük dilimi kapladığını görsel olarak ortaya koyuyor.

6.4. Bekleme Süresi Dağılımı

Her paket türünün ortalama bekleme süresini karşılaştıran bir bar chart ekledik. Böylece öncelik mekanizmasının bekleme süreleri üzerindeki etkisi doğrudan görülüyor.

SUMO gibi bir ağ simülasyon aracı kullanmadık çünkü SUMO daha çok araç trafiği için tasarlanmış. Bizim ağ trafiği senaryomuz için SimPy + Streamlit + Plotly kombinasyonu yeterli oldu. İleride daha karmaşık bir topoloji modellenenecekse ns-3 gibi bir araca geçiş düşünülebilir.

7. Sonuç

Projeyi geliştirirken en çok faydalandığımız şey, teorik olarak bildiğimiz kavramları gerçekten çalışır halde görmek oldu. Mesela "öncelik kuyruğu düşük öncelikli

paketleri geriye atar" diye biliyorduk ama bunu simülasyonda rakamlarla görmek başka bir şey. Admin paketlerinin gerçekten sıfır kayıpla geçtiğini, şüpheli paketlerin yaklaşık %39 oranında drop edildiğini gözlemledik.

Tabii modelin sınırlılıkları var. Gerçek bir güvenlik duvarı sadece öncelik değerine bakarak karar vermez; derin paket inceleme, IP itibar skoru gibi çok daha karmaşık mekanizmalar devrededir. Ama bir benzetim dersi projesi olarak, temel kavramları doğru modelleyebildiğimizi düşünüyoruz.

Kodun tamamına ve daha detaylı dokümantasyona GitHub üzerinden ulaşılabilir:

<https://github.com/emircanaltuntas/GateKeeper>

Kaynakça

[1] SimPy Documentation — <https://simpy.readthedocs.io>

[2] Matloff, N. (2008). Introduction to Discrete-Event Simulation and the SimPy Language.

[3] Stallings, W. (2017). Network Security Essentials, 6th Edition, Pearson.

[4] Gross, D., Shortle, J.F., Thompson, J.M., Harris, C.M. (2008). Fundamentals of Queueing Theory, 4th Edition, Wiley.

[5] Streamlit Documentation — <https://docs.streamlit.io>

[6] Plotly Python Documentation — <https://plotly.com/python/>